IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 15 December 2025, accepted 2 February 2026, date of publication 10 February 2026, date of current version 27 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3663383

RESEARCH ARTICLE

A Self-Adaptive Framework for Deploying Machine Learning Systems Without Ground-Truth Data at Runtime

KENTO FURUKAWA[1], HIROYUKI NAKAGAWA[1,2], (Member, IEEE), AND TATSUHIRO TSUCHIYA[1], (Member, IEEE)

[1]Graduate School of Information Science and Technology, Osaka University, Suita, Osaka, Japan
[2]Graduate School of Environmental, Life, Natural Science and Technology, Okayama University, Okayama, Japan

Corresponding author: Hiroyuki Nakagawa (nakagawa@ist.osaka-u.ac.jp and h-nakagawa@okayama-u.ac.jp)

ABSTRACT In recent years, the practical application of machine learning technology has rapidly progressed, accelerating its adoption across various fields. In this context, studies into the effective operation of machine learning systems in real-world environments have become essential. In actual operational settings, the distribution of input data often changes over time, leading to a significant decline in the predictive performance of models. Additionally, the lack of ground-truth data for test data during operation can sometimes make adaptation through retraining difficult. This study proposes a framework that autonomously adapts to changes in input data distribution, even in environments where ground-truth data for test data is unavailable during operation. This framework analyzes the distribution of input data and selects the appropriate predictive model based on the state of the distribution. To ensure optimal model selection, the framework employs two complementary approaches: 1) dynamically switching between multiple pre-trained models with different feature sets according to environmental changes and 2) building ensemble models based on the distribution of the test data. These approaches enable the framework to autonomously adapt to shifts in data distribution, even in operational settings where ground-truth data is unavailable. Evaluation experiments using both simulated and real-world data assessed the predictive performance of the proposed method through metrics such as R^2, RMSE, and MAE. Compared to conventional single model predictions, the proposed method consistently demonstrated higher accuracy. These results indicate that the proposed approach effectively adapts to data distribution shifts in operational environments where ground-truth data is unavailable.

INDEX TERMS Self-adaptive systems, frameworks, machine learning.

I. INTRODUCTION

In recent years, technological advancements in machine learning (ML), particularly in deep learning, have rapidly facilitated the development of innovative ML-enabled systems (MLS) such as ChatGPT, Google Bard, and DALLE-2. These systems play a significant role in decision support and predictive analytics within industries, including financial market analysis, medical diagnostics, and manufacturing process optimization. However, the implementation and operation of MLS pose complex software engineering challenges, including model version control and data quality assurance. Specifically, the development and integration processes of ML components are recognized as critical factors directly affecting the reliability and performance of the entire system.

When deploying MLS in real-world environments, many projects face significant challenges during the transition to production. There are several environmental factors that significantly impact system performance metrics, such as fluctuations in model prediction performance, instability in the underlying software components, variations in

The associate editor coordinating the review of this manuscript and approving it for publication was Chuan Heng Foh.

computational resource costs, changes in power consumption, and sudden surges in system request volumes. A particularly notable issue is the severe decline in model performance caused by temporal changes in the statistical distribution of input data. This phenomenon arises from a combination of factors, including sudden market trend shifts, regional variations, irregular behaviors in data collection and processing pipelines, and changes in user behavior patterns.

These variations in input data distribution are formally defined in the field of machine learning as *covariate shift*, a fundamental cause of model performance degradation. Previous studies on covariate shift have mathematically analyzed the impact of discrepancies between training and test data distributions on predictive performance. These studies have proposed methodologies such as importance weighting and transfer learning as countermeasures. However, the majority of existing studies assume static environments and lacks sufficient investigation into adaptive mechanisms for dynamic environmental changes during real-world operation.

Recently, *self-adaptive* technology has emerged as a promising approach to addressing these challenges. This technology continuously monitors the system's operational state and autonomously adjusts its structure and behavior in response to changes in performance metrics or functional requirements. Self-adaptive framework has already demonstrated effectiveness in fields such as Cyber-Physical Systems of Systems (CPSoS) and the Internet of Things (IoT) [1], [2]. Nevertheless, studies on applying this technology to MLS remain limited, and there are no established design guidelines for adaptive mechanisms that address the unique challenges of real-world environments.

Previous studies [3], [4] have proposed methods for autonomously adapting to environmental changes in operational classification tasks; however, these methods assume that ground-truth data for test data can be obtained during system operation. In practice, after deploying machine learning systems, it is often challenging to acquire ground-truth data during operation due to time constraints or ethical barriers.

To address this issue, this paper proposes a framework that autonomously adapts to changes in data distribution and prevents performance degradation, even when ground-truth data is unavailable during operation. In such scenarios, where retraining the model using the latest test data to adapt to data distribution shifts is infeasible, the proposed approach analyzes the distribution of input data and dynamically adjusts the predictive model accordingly. To select the most suitable model, the framework employs two adaptive tactics: an *model switching tactic* and an *ensemble model building tactic*. The model switching tactic involves selecting and switching to the most appropriate model from a set of pre-trained models. In contrast, the ensemble model building tactic dynamically builds a new ensemble model by combining predictions from multiple pre-trained models to better align with the current data distribution.

By leveraging these adaptive tactics based on the state of the input data distribution, the proposed framework enables autonomous adaptation to distribution shifts, even in situations where ground-truth data for test data is unavailable during operation.

To rigorously evaluate the effectiveness of the proposed approach, we conducted comprehensive experiments on regression tasks using both simulation and real-world data. The experiments compared the predictions from the proposed approach with those from traditional single models, using evaluation metrics such as the coefficient of determination (R^2), mean absolute error (MAE), and root mean square error (RMSE), ensuring a multifaceted performance evaluation.

The results showed that the proposed approach outperformed single models across all evaluation metrics. These results validate the effectiveness of the proposed approach in adapting to input data distribution shifts in real-world environments where ground-truth data is difficult to obtain during operation.

II. RELATED WORKS

A. SELF-ADAPTIVE SYSTEMS

Self-adaptive Systems are a paradigm designed to autonomously adjust system behavior in uncertain environments, ensuring the maintenance of expected quality attributes [5], [6].

The importance of this adaptability has grown significantly in modern software engineering, where increasing system scale and complexity necessitate addressing uncertainties that cannot be anticipated during design [7].

Against this backdrop, the MAPE-K feedback loop was proposed as a framework for realizing self-adaptive systems [8], [9], [10]. The MAPE-K feedback loop is a systematic approach for achieving adaptive control in self-adaptive systems, comprising five key phases: Monitor, Analyze, Plan, Execute, and Knowledge. Each phase has a distinct role and interacts with the others to ensure continuous system adaptation. The Monitor phase collects data on the system's internal state and external environmental conditions. In the Analyze phase, the data received from the Monitor phase is analyzed to diagnose the system's current state, assess the need for adaptation, and trigger the Plan phase when necessary. The Plan phase develops specific adaptation tactics based on the results of the Analyze phase and selects the optimal action from available options to achieve the system's goals. Next, the Execute phase implements the adaptation tactics defined in the Plan phase. The Knowledge phase stores static information about the system, such as its structure, operational policies, goals, and constraints, while maintaining dynamic information, including past adaptation histories, learned knowledge, and runtime state data. This information supports decision-making and facilitates coordination among the phases. By continuously cycling through these phases, the MAPE-K feedback loop enables systems to

autonomously adapt to changing conditions, ensuring long-term reliability and effectiveness.

The effectiveness of self-adaptive systems has been demonstrated in applications within fields such as IoT systems [11], [12], [13] and Cyber-Physical Systems of Systems (CPSoS) [1], [14], [15], [16], [17]. In these domains, systems have successfully maintained stability and efficiency by autonomously adapting to uncertainties. A notable IoT example is DeltaIoT, a self-adaptive system deployed in IoT networks for applications like building security surveillance and smart factory monitoring [18]. DeltaIoT addresses uncertainties such as communication interference, traffic load fluctuations, and battery depletion. By leveraging the MAPE-K loop, it collects and analyzes data on transmission power, communication delays, and packet loss rates from individual motes, optimizing energy efficiency and communication performance.

In the domain of CPSoS, a representative example is Platooning LEGOs, a system simulating vehicle platooning with autonomous cars [19]. Each vehicle operates as an independent system but collaborates to achieve higher-level objectives such as improving fuel efficiency and minimizing road occupancy. Using LEGO MINDSTORMS EV3 vehicles, this system adapts to environmental uncertainties through sensor data and inter-vehicle communication, achieving goals like collision avoidance and platoon maintenance. Particularly noteworthy is its capability to adapt to challenges unique to physical environments, such as sensor noise, communication delays, and physical inaccuracies, which are difficult to replicate in simulations.

Thus, self-adaptive systems have been demonstrated to play a vital role in modern complex software systems. They are gaining attention as a promising approach for maintaining and improving system quality attributes, particularly in environments where uncertainties cannot be anticipated during the design phase. Through applications in IoT networks and cyber-physical systems, self-adaptive mechanisms have shown significant contributions to enhancing system stability, efficiency, and reliability. Furthermore, their effectiveness is being validated in scenarios where multiple systems need to operate collaboratively. These achievements highlight the development of self-adaptive systems as a critical research area in contemporary software engineering.

B. MACHINE LEARNING TECHNIQUES FOR SELF-ADAPTIVE SYSTEMS

In recent years, the application of machine learning techniques in self-adaptive systems has grown significantly, with studies advancing in a wide range of fields, including cloud services, cyber-physical systems (CPS), IoT, and robotics [20], [21]. Self-adaptive systems are characterized by their ability to autonomously adjust and optimize themselves to maintain objectives in response to changes in external environments and internal states. This adaptability to dynamic situations is essential. However, traditional rule-based adaptation and fixed feedback control mechanisms have limitations in addressing unpredictable events and environmental variations. To overcome these challenges, the incorporation of machine learning has become increasingly important. Among the primary application areas for machine learning in self-adaptive systems are cloud computing, CPS, and client-server systems, where studies have been particularly prominent.

The adaptation challenges in cloud computing, CPS, and client-server systems, as well as the application of machine learning to address these challenges, vary according to the specific characteristics of each domain. In cloud computing, efficient resource allocation in response to dynamic workload changes is critical. Maintaining quality properties while minimizing operational costs represents a central adaptation challenge. Machine learning techniques, such as resource usage prediction and dynamic policy updates, have proven to be effective in addressing these issues [22], [23], [24].

In CPS, the seamless integration of physical environments and digital systems necessitates reliability and performance enhancements through quality management, efficient resource utilization, anomaly detection, and strengthened cybersecurity. Machine learning plays a vital role in addressing these challenges by enabling runtime model updates and anomaly detection capabilities [25], [26].

For client-server systems, the primary adaptation challenge involves quality management to reduce latency and improve reliability while responding to dynamic workload fluctuations. Additionally, minimizing operational costs while reducing service failure rates and ensuring efficient CPU and memory utilization are critical requirements. Machine learning techniques, such as reinforcement learning for dynamic policy updates and resource demand forecasting, are widely employed to tackle these challenges [27].

While the discussion thus far has focused on machine learning applications in various domains, this section shifts attention to the functional components of the MAPE-K loop and examines the specific phases where machine learning is most actively utilized. The MAPE-K loop comprises four primary phases: Monitoring, Analysis, Planning, and Execution, each of which plays a crucial role in controlling the operations of self-adaptive systems. Among these, the Analysis and Planning phases stand out as the most extensively studied and represent the areas where machine learning applications have advanced the most.

In the Analysis phase, machine learning is utilized to reduce the adaptation space and rank adaptation options [28], [29]. This phase identifies the most relevant adaptation choices based on environmental changes while excluding irrelevant options, enabling more efficient analysis. Consequently, systems can respond rapidly and effectively to dynamic situations.

In the Planning phase, machine learning supports the selection of optimal adaptation tactics and the optimization of planning processes [30]. This phase leverages machine

learning to dynamically adapt to environmental changes, thereby improving the system's flexibility and efficiency. Techniques such as instance-based learning and other optimization approaches are employed to enhance planning processes.

In summary, the application of machine learning across the phases of the MAPE-K loop provides a critical foundation for enabling self-adaptive systems to effectively respond to dynamic environmental changes.

C. SELF-ADAPTIVE FRAMEWORK FOR MACHINE LEARNING SYSTEMS

In recent years, while studies on applying machine learning (ML) techniques to self-adaptive systems have advanced significantly, studies on incorporating self-adaptive techniques into ML systems themselves to make them self-adaptive remain underdeveloped. Many ML systems are trained under the assumption of static data distributions, meaning that performance can degrade significantly when data distributions change during deployment. For instance, input data characteristics may change unexpectedly due to market trend shifts, geographical differences, or errors in sensors and data pipelines. Under such circumstances, maintaining a single pre-trained model often proves inadequate.

Therefore, it is essential to leverage self-adaptive techniques to enable ML systems to autonomously respond to environmental changes and continuously maintain or improve performance. Self-adaptive systems have the capability to monitor changes in their internal state and external environment in real time and to automatically adjust or adapt accordingly. By integrating self-adaptive mechanisms into ML systems, resilience to environmental fluctuations can be enhanced, ensuring stable performance, as well as improving operational efficiency and reliability.

From a broader perspective, research on adaptive systems has been conducted in other fields as well. In control systems, adaptive switching supervisory control has gained attention as a promising technology for handling uncertain plants. Multi-model falsification-based unfalsified adaptive switching supervisory control (UASSC) frameworks have been proposed [31], and UASSC theory has been extended to noisy environments [32]. These methods achieve adaptation through switching among multiple candidate controllers.

In the field of multi-model adaptive control, approaches based on integration and coordination of multiple models have also been proposed. Cooperative utilization of multiple identification models with time-varying convex combination estimation dependent on the collective output of all models has been developed [33]. Additionally, adaptive mixing control (AMC) for discrete-time systems has been proposed, achieving smooth mixing of multiple candidate controllers to avoid abrupt switching [34].

Similarly, in the machine learning field, previous studies have aimed to construct frameworks enabling ML-based systems to function adaptively in dynamic and unpredictable environments [35], [36]. These studies have focused on addressing the issue of degraded ML component performance due to environmental changes or data quality deterioration, which can compromise the overall utility of the system. To tackle this challenge, discussions have been held on extending the adaptation processes (e.g., the MAPE-K loop) traditionally employed in self-adaptive systems and incorporating mechanisms tailored to the specific characteristics of ML systems.

For example, a self-adaptive framework has been proposed for object detection tasks with fluctuating traffic volumes to maintain quality of service (QoS) [37]. This framework utilizes various versions of YOLOv5 (YOLOv5n, YOLOv5s, YOLOv5m, YOLOv5l, YOLOv5x), each offering distinct trade-offs between processing speed and accuracy due to differences in parameter count and structure.

For instance, YOLOv5n is lightweight and fast but slightly less accurate, whereas YOLOv5x has more parameters, achieving higher accuracy but at a slower processing speed. By dynamically selecting the optimal model based on real-time traffic conditions, the system ensures optimal performance. The selection process adheres to QoS requirements, choosing the most accurate model that meets the required processing speed at any given moment.

When traffic increases and the system experiences high load, a model prioritizing processing speed (e.g., YOLOv5n) is selected. Conversely, when the load decreases, a model prioritizing accuracy (e.g., YOLOv5x) is employed. This dynamic switching mechanism continuously optimizes system performance and responsiveness.

However, for other ML tasks, such as regression or classification, simply switching models based on a trade-off between processing speed and accuracy is often insufficient. These tasks typically involve more complex environmental changes, where data distributions or input characteristics may shift unpredictably. Therefore, switching between pre-trained models alone may not adequately address these challenges.

A self-adaptive framework has also been proposed to address the issue of mispredictions in ML models deployed in real-world contexts [3]. During deployment, ML models may encounter data distributions that differ from those seen during training, leading to mispredictions. Such mispredictions can negatively impact system performance and utility, resulting in violations of service level agreements (SLAs) and increased business costs. The proposed framework employs probabilistic model checking to estimate the expected performance improvements from retraining and assess the impact of adaptation on system utility. This approach efficiently manages trade-offs between performance improvements and execution costs, enhancing decision-making for the system.

A subsequent study has proposed frameworks that consider delayed ground-truth data availability during operation [4]. While earlier studies assumed immediate access to labels for performance evaluation, more recent work has introduced

frameworks that account for delayed access to ground-truth data.

However, these studies still assume the availability of ground-truth data for performance evaluation and retraining during operation. Scenarios where ground-truth data cannot be obtained during operation have not been adequately addressed. Additionally, the previous study [37] primarily focused on model switching as the sole recovery action for adapting to distribution shifts, while others [3], [4] emphasized retraining as the only solution, resulting in limitations in adaptive capabilities.

Addressing these challenges, our study proposes a framework that eliminates the dependency on ground-truth data during operation. Both control theory approaches and existing ML studies require some form of performance feedback. Control systems evaluate performance using system response metrics, while ML systems use ground-truth data for performance evaluation. In contrast, our proposed framework operates without ground-truth data during operation. Specifically, we propose a self-adaptive framework capable of adapting to shifts in input data even when ground-truth data is unavailable during operation. Furthermore, by combining multiple adaptation tactics—namely, ''model switching'' and ''ensemble model building''—our framework can adapt to complex changes in data distributions, ensuring robust and efficient system performance.

III. SELF-ADAPTIVE FRAMEWORK WITHOUT GROUND-TRUTH DATA DURING OPERATION

In this study, we propose a framework to address changes in the input data distribution in machine learning system operational environments and prevent prediction accuracy degradation. In real-world operational settings, the input data distribution of machine learning models often changes. Failure to properly adapt to these distribution shifts can lead to significant performance degradation, undermining the reliability of machine learning systems. Machine learning problems are generally formulated under the assumption that training and test data are independently sampled from the same distribution. Based on this assumption, learning methods that minimize errors in training data have been established, and when good results are obtained on training data, similar performance is expected on test data. However, in many real-world problems, this assumption does not hold due to biases and distribution shifts. Consequently, training methods that minimize errors on the training data may learn spurious correlations, leading to performance degradation in different environments. Thus, continuing to use a pre-trained single model in the presence of data distribution shifts often results in reduced prediction accuracy. Furthermore, when ground-truth data is unavailable during operation, it becomes challenging to adapt to data distribution shifts by retraining on data from the new environment.

To overcome these challenges, this study proposes a framework that dynamically selects the optimal model according to environmental changes. The proposed framework consists of two phases: a training phase and an operational phase. In the training phase, models are trained, and the system requirement is defined. During model training, multiple models are created. By training models with different feature sets, a model group is constructed to adapt to input data distribution shifts during operation. Additionally, the system requirement, such as the performance requirement based on the width of the prediction interval, is defined in this phase.

During the operational phase, as illustrated in Algorithm 1, the four phases (Monitor, Analyze, Plan, Execute) are cyclically executed for continuous data streams. In the Monitor phase, newly arriving data samples are accumulated in a buffer while continuously monitoring the test data (Algorithm 1, lines 6-7). In the Analyze phase, feature variations are detected using the accumulated test data as input (line 8). Importantly, a detected drift state is confirmed only when it persists for a certain period, thereby preventing misclassification due to temporary fluctuations (lines 9-14).

In the Plan phase, the optimal model is selected based on the confirmed drift state (lines 15-25). When no drift is detected, the base model is employed (lines 16-17). Upon drift confirmation, a hierarchical model selection strategy is applied. First, the base model's prediction interval is evaluated (lines 18-19), followed by evaluation of feature-excluded models' prediction intervals. The first model whose prediction interval width falls below the threshold is adopted (lines 20-21). If no model satisfies the threshold criterion, ensemble model construction is performed (lines 22-23). By evaluating model uncertainty based on prediction interval width, appropriate model selection becomes feasible during operation when ground-truth data is unavailable. In the Execute phase, predictions are generated using the selected model and the results are output (line 26).

Thus, by combining two adaptation tactics—a simple switch to a pre-trained model and the construction of an ensemble model using multiple models—the framework minimizes the impact of input data distribution shifts on prediction accuracy and ensures stable system performance even in situations where ground-truth data is unavailable during operation.

The introduction of this framework enables the realization of machine learning systems that autonomously adapt to diverse input data distribution shifts during operation. This approach achieves efficient system operation and prevents prediction accuracy degradation, even in situations where ground-truth data is unavailable during operation, thereby enhancing the reliability and practicality of machine learning systems.

A. TRAINING PHASE

1) MODEL BUILDING

During the training phase, multiple models are prepared as a foundation for building a machine learning system capable

Algorithm 1 Self-Adaptive Framework

Require: X: test data stream; *Models*: pre-trained models; θ: PI threshold; N: stability threshold
Ensure: *Predictions*: prediction results

$State \leftarrow initializeState()$
$State.drift\_set \leftarrow \emptyset$
$State.counter \leftarrow 0$
$State.buffer \leftarrow []$
while X has next sample **do**
  $x_{new} \leftarrow X.next()$ **// Monitor**
  $updateBuffer(State, x_{new})$
  $drift \leftarrow detectDrift(State.buffer)$ **// Analyze**
  if $drift \neq State.drift\_set$ **then**
    $State.drift\_set \leftarrow drift$
    $State.counter \leftarrow 1$
  else
    $State.counter \leftarrow State.counter + 1$
  end if
  if $State.counter \geq N$ **then**
  // Plan
  if $State.drift\_set = \emptyset$ **then**
    $model \leftarrow base\_model$
  else if $PI(base\_model) \leq \theta$ **then**
    $model \leftarrow base\_model$
  else if $\exists\, excluding\_model : PI(excluding\_model) \leq \theta$ **then**
    $model \leftarrow excluding\_model$
  else
    $model \leftarrow buildEnsemble(Models, State.buffer)$
  end if
  end if
  $Predictions.append(predict(model, x_{new}))$ **// Execute**
end while
return *Predictions*

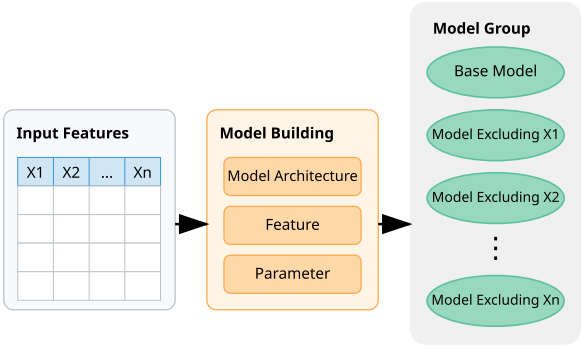

FIGURE 1. The flow of building multiple models.

of flexibly adapting to variations in data distribution during deployment. In this phase, the model architecture is first selected, and for each model, both the selection of features and parameter tuning are conducted. This ensures that each model is individually optimized to achieve its best possible performance.

Specifically, as shown in Figure 1, in addition to a base model that utilizes all features, models excluding each individual feature are trained. First, the data available for training is divided into the training dataset and validation dataset, and each model is trained using the training dataset. The base model, which incorporates all features, is designed to maximize predictive accuracy in environments where the input data distribution does not deviate significantly from the training data.

Furthermore, to enable model switching during deployment, models excluding specific features are built during the training phase. These models are expected to serve as alternatives in cases where certain features are missing or affected by noise or bias in the deployment environment. By building multiple models that differ in the features used for training, the system can dynamically select the most suitable model based on the state of the data during deployment.

Through such preparations in the training phase, a foundation is established for the system to dynamically select and switch models in response to environmental changes, such as variations or missing features in the input data distribution during deployment. This ensures the realization of a predictive system that is both flexible and robust, even under changing operational conditions.

2) SYSTEM REQUIREMENT DEFINITION

In the training phase, a system requirement is also established. Specifically, a performance requirement based on the width of the prediction interval is defined. The width of the prediction interval is commonly used as an indicator for assessing the uncertainty in model predictions.

To define the system requirement, the system first calculates the average width of the prediction interval for each sample in the validation data (PI_{val}) using a base model trained with all features. Since the validation data follows the same distribution as the training data, PI_{val} serves as the baseline for predictive uncertainty under conditions where the data distribution remains stable.

The system then multiplies PI_{val} by a user-specified tolerance parameter k to establish an upper limit, PI_{max}. By defining the system requirement that "the width of the prediction interval during deployment must not exceed PI_{max}," the system provides a mechanism for evaluating whether adaptive measures are necessary when this threshold is exceeded.

During operation scenarios where ground-truth data is unavailable, monitoring performance metrics such as R^2 or RMSE is not feasible. However, by defining the system requirement based on the width of the prediction interval, the system can evaluate predictive uncertainty even in the absence of ground-truth data during operation.

By setting PI_{max} as a system requirement, the system can use it to determine whether a model switch is needed or as a basis for model selection during deployment.

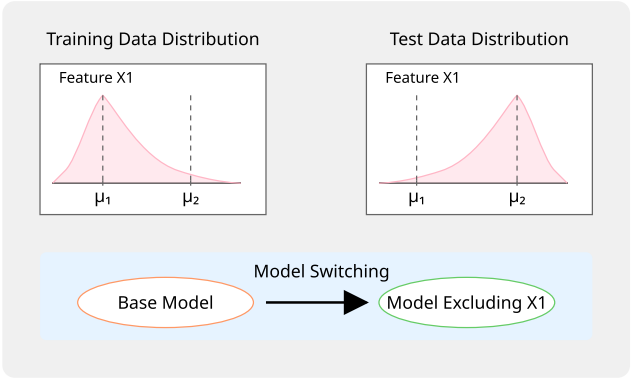

FIGURE 2. Conceptual diagram of model switching.

B. OPERATIONAL PHASE

1) ADAPTIVE TACTICS

a: MODEL SWITCHING

The generalization error of a machine learning model can be decomposed into three components: bias, variance, and irreducible noise. When a specific feature experiences covariate shift during deployment, where the test distribution diverges significantly from the training distribution, models incorporating this feature suffer from inflated prediction variance. This occurs because the relationships learned during training no longer apply to the shifted distribution, causing predictions to fluctuate unstably in response to variations in the affected feature. In stable environments, including more features typically reduces bias by enriching the model's representational capacity. However, when covariate shift occurs, the affected feature contributes more noise than signal, and the resulting variance inflation outweighs any gains from bias reduction. Excluding the shifted feature removes this source of instability, thereby reducing prediction variance. Although this simplification increases bias due to reduced model capacity, the magnitude of this increase is limited. Consequently, the variance reduction dominates the bias increase, ultimately lowering the overall generalization error.

Based on this theoretical foundation, the model switching tactic ensures prediction stability by selecting the most suitable model from a pre-prepared set of models based on changes in the input data distribution. Specifically, the system trains a ''base model'' using all available features and several ''feature-excluded models,'' which are trained by excluding specific features. Depending on the state of the input data distribution, the most appropriate model is selected for prediction.

The base model utilizes all features to make predictions. When there are no significant changes in the input data distribution, all features contribute valuable information, allowing the base model to achieve the highest predictive accuracy. Therefore, the system initially attempts to use the base model, based on the assumption that leveraging all available information results in more accurate predictions.

However, if changes are detected in the input data distribution, the affected features can be detrimental to the predictive accuracy as they introduce increased variance into predictions. In such cases, as shown in Figure 2, it is effective to use feature-excluded models trained without the features influenced by the distributional changes. For example, if the distribution of feature X1 undergoes a significant shift, the model trained without X1, referred to as ''model excluding X1'', can make predictions using only stable features unaffected by the change. This approach mitigates the negative impact of distributional changes on predictive accuracy by dynamically optimizing the bias-variance tradeoff.

The effectiveness of this tactic lies in its ability to select the appropriate set of features based on the state of the input data distribution. When the input data distribution shows no significant shifts, the base model, which utilizes all features, provides high predictive accuracy since all features offer valuable information. Conversely, when changes in the input data distribution are detected, using feature-excluded models that exclude affected features helps maintain stable predictions. By explicitly removing features impacted by the distributional shift, this approach effectively suppresses the adverse effects of such changes on predictive accuracy.

In this way, the model switching tactic represents an effective approach to maintaining prediction stability by selecting the appropriate model based on the state of the input data distribution. The base model is used when all features are beneficial, while feature exclusion models are selected when specific features should be omitted. This flexibility allows the system to adapt to a variety of input data distribution shifts.

b: ENSEMBLE MODEL BUILDING

The objective of the ensemble model building tactic is to adapt to complex variations in the input data distribution that cannot be handled by switching to an existing model. As shown in Figure 3, this tactic builds an ensemble model using stacking through a three-step process. Stacking is a method that leverages the output values of multiple predictive models as new features and integrates them using a meta-model to make the final prediction [38]. In this tactic, the first step is to select samples from the training and validation data that are most similar to the test data distribution. For this purpose, Mahalanobis distance is used to measure similarity. The feature matrix of the combined training and validation data is denoted as X_{combined}, and its corresponding target variable as y_{combined}. The feature matrix of the most recent M test data samples is denoted as X_{test}. All features are used for the Mahalanobis distance calculation. Next, the covariance matrix Σ of X_{combined} is computed as follows:

$$\Sigma = \text{Cov}(X_{\text{combined}})$$

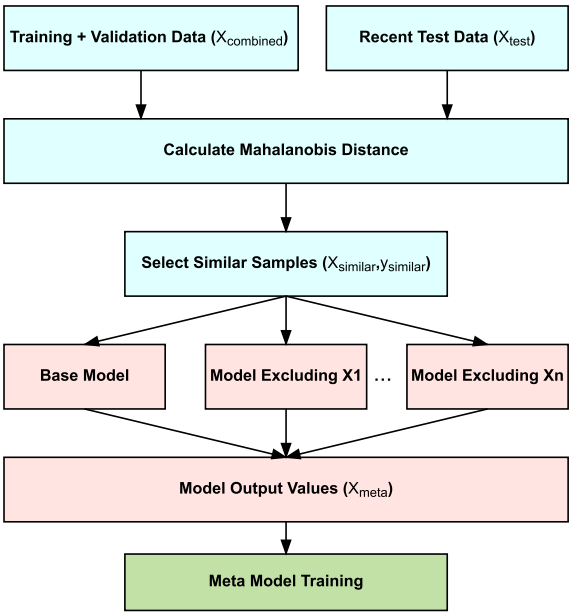

FIGURE 3. Ensemble model building process.

Using this covariance matrix, the Mahalanobis distance between each data point $x_{\text{combined},j}$ in the combined data and each test data point $x_{\text{test},i}$ is defined as:

$$d_M(x_{\text{combined},j}, x_{\text{test},i}, \Sigma) = \sqrt{(x_{\text{combined},j} - x_{\text{test},i})^\top \Sigma^{-1} (x_{\text{combined},j} - x_{\text{test},i})}$$

The Mahalanobis distances for all j, i pairs are calculated, and the average distance for each data point $x_{\text{combined},j}$ to all test data points is computed as follows:

$$\text{mean\_distance}(x_{\text{combined},j}) = \frac{1}{M} \sum_{i=1}^{M} d_M(x_{\text{combined},j}, x_{\text{test},i}, \Sigma)$$

where M is the number of test data samples. The set of indices I_s below corresponds to the top s indices when these mean distances are arranged in ascending order:

$$I_s = \{j_1, j_2, \ldots, j_s\}$$

Subsequently, the s samples most similar to the test data are defined as:

$$X_{\text{similar}} = \left\{ x_{\text{combined},j} \mid j \in I_s \right\},$$

Similarly, the corresponding target variables are defined as:

$$y_{\text{similar}} = \left\{ y_{\text{combined},j} \mid j \in I_s \right\}$$

These similar samples, $(X_{\text{similar}}, y_{\text{similar}})$, are then used in the next step to obtain the output values of the predictive models.

Next, the output values of multiple predictive models are obtained using the selected samples X_{similar}, which are treated as new features. The output values for X_{similar} are calculated using the model trained on all features. Similarly, output values are calculated for each model trained with one feature excluded. These output values are concatenated column-wise to form a new feature matrix X_{meta}.

Finally, the meta-model is trained using the constructed X_{meta}. The meta-model is not restricted to a specific method and can be chosen based on the task, such as linear regression or nonlinear models. The meta-model is trained to take X_{meta} as input and predict y_{similar}.

This tactic realizes an adaptive model building approach by dynamically building the meta-model based on the test data distribution at runtime. Unlike conventional stacking methods, which build a static meta-model using the entire training data during the learning phase, this tactic dynamically generates a meta-model tailored to the distribution characteristics of the test data at the time of prediction. Furthermore, the meta-model training utilizes the output values of both the model trained on all features and the models trained with each feature excluded. This ensures that even if the distribution of a specific feature shifts, other models can compensate for its impact, mitigating the biases and weaknesses of individual models. Additionally, by optimally integrating the outputs of multiple models, the meta-model enables optimal weighting under changing data distributions, allowing for predictions that account for multiple distributional shift factors.

In summary, building an ensemble model in this manner enables effective adaptation to complex input data distribution shifts, even in situations where ground-truth data is unavailable during operation. Since sample selection and model training are performed based solely on the input data distribution characteristics, adaptive predictions can be made without relying on the presence of ground-truth data during operation. Moreover, leveraging diverse models ensures stable predictive performance even during distributional shifts. This enables the realization of an autonomous adaptation tactic for addressing input data distribution shifts in operational environments where ground-truth data is difficult to obtain.

2) ARCHITECTURE OF THE MAPE-K LOOP

To enable a machine learning system to autonomously adapt to changes in data distribution during deployment, the MAPE-K loop is employed. The MAPE-K loop is a fundamental framework for self-adaptive systems, describing a process in which the system monitors its environment in real time, makes appropriate decisions, and dynamically adapts to changing conditions. The loop consists of five elements: Monitor, Analyze, Plan, Execute, and Knowledge. This framework allows the system to understand its state and respond effectively to uncertainties and variations encountered during operation. Figure 4 illustrates the architecture of the MAPE-K loop in the proposed method. Below, we provide a detailed explanation of the roles and implementation of each component in this approach.

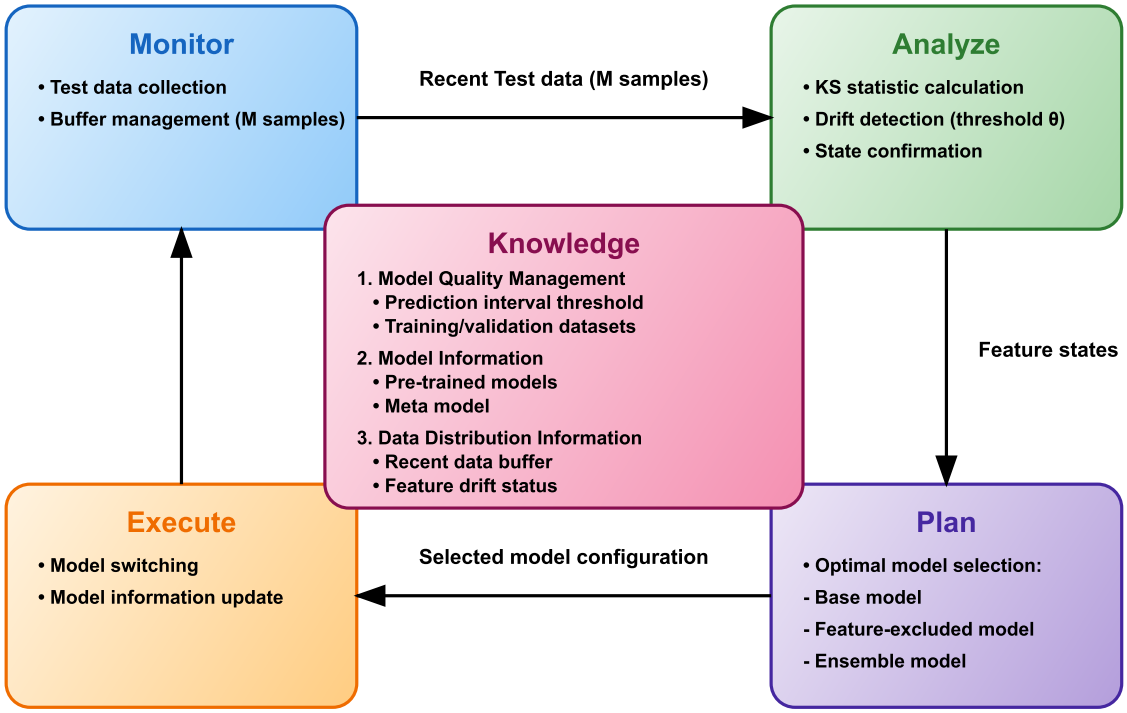

FIGURE 4. Architecture of the MAPE-K loop.

a: KNOWLEDGE

In the Knowledge phase, the data necessary for system operations is stored. The information managed and utilized in this system is categorized into three groups, each serving a specific role. Model quality management information includes data that forms the foundation for maintaining the predictive quality of the system. For example, threshold values for the width of the prediction interval, as part of the system requirement, function as benchmarks to assess prediction reliability. Training and validation datasets are used as reference data for evaluating model performance and detecting distributional changes, making this information a critical basis for ensuring the system's overall predictive quality.

Model information manages the core data required for executing actual predictive operations. It includes various pre-trained models, which are selected and applied based on their suitability to the current environment. Additionally, when ensemble learning is employed, the system manages meta model information. This information constitutes the backbone of the system's predictive capabilities.

Data distribution information supports the system's adaptive monitoring functionality. It retains the latest data as a buffer and continuously monitors distributional differences from the training data. It detects and records shifts for each feature and tracks their persistence over time. These insights serve as critical decision-making inputs for selecting appropriate models in response to changes in data distribution.

These types of information are interrelated and are continuously utilized and updated throughout each phase of the system's operational loop.

b: MONITOR

In the Monitor phase, test data from the operational environment is collected. The collected data is subsequently used in the Analyze phase to assess the data distribution.

c: ANALYZE

The Analyze phase plays a crucial role in detecting changes in data distribution during system operation, determining the appropriate timing for model selection, and providing the Plan phase with essential information regarding distributional changes necessary for selecting the optimal model.

To detect data drift, this phase evaluates the distributional differences for each feature between the most recent operational data (the latest M samples) and the training data using the Kolmogorov-Smirnov (KS) statistic. This statistic quantitatively measures the disparity between two distributions, and a feature is considered to exhibit a distributional change if the KS statistic exceeds a threshold, θ. This approach enables continuous monitoring of distributional changes for each feature.

However, detected distributional changes are not immediately confirmed as system states. A change is recorded as a new confirmed state only if it is consistently detected for the same combination of features over N consecutive

evaluations. This design, which emphasizes the persistence of changes, helps distinguish between transient noise and genuine distributional shifts.

Once a confirmed distributional change is recorded, the system registers it as a new state and initiates the model re-selection process. Specifically, upon confirmation of the change, the system transitions to the Plan phase to select the optimal model based on the detected changes in feature distribution. Conversely, if no distributional change is confirmed, the system continues to use the current predictive model to maintain stability in its predictions.

In this way, the Analyze phase fulfills a vital control function by statistically evaluating data changes, verifying their persistence, and triggering the model adaptation process when necessary. Although it is not possible to analyze the progression of performance metrics such as R^2 or RMSE due to the lack of ground-truth data during operation, the Analyze phase compensates by evaluating shifts in input data distribution. This enables the system to trigger the model adaptation process and provides the Plan phase with critical information about the state of data distribution, which is essential for model selection.

d: PLAN

The Plan phase is a critical stage in selecting the most suitable model to adapt to changes in input data distribution. By employing two adaptive tactics, namely model switching and ensemble model building, the Plan phase determines the optimal model based on the state of the data distribution.

In the Plan phase, the system first receives information from the Analyze phase regarding detected distribution shifts in specific features. If no shifts are detected in any features, the system adopts the model switching tactic and selects the base model, which is trained using all features.

When distribution shifts are detected in certain features, the system initially attempts to adapt using the model switching tactic. Specifically, it begins by evaluating the width of the base model's prediction interval. This evaluation uses the most recent M test data samples. If the average width of the prediction interval, $\mathrm{PI}_{\mathrm{base}}$, is below a predefined threshold $\mathrm{PI}_{\mathrm{max}}$ as the system requirement, the base model is selected, as it meets the requirement:

$$\mathrm{PI}_{\mathrm{base}} \leq \mathrm{PI}_{\mathrm{max}}$$

If the base model does not meet this requirement, the system evaluates models trained without each shifted feature to determine if they satisfy the threshold. For each model trained with an excluded feature, the average width of the prediction intervals, $\mathrm{PI}_{\mathrm{excluding}X_i}$, is calculated, and it is evaluated to determine whether the following condition is satisfied:

$$\mathrm{PI}_{\mathrm{excluding}X_i} \leq \mathrm{PI}_{\mathrm{max}}$$

If multiple models meet this condition, the system selects the model with the smallest average width of prediction intervals.

This tactic provides the advantage of rapid adaptation by appropriately switching between existing models without requiring additional training.

If the model switching tactic fails to meet the system requirement, the system transitions to a more advanced tactic: ensemble model building. This tactic combines training data, validation data, and the most recent M test data samples to build a stacking-based meta-model. Specifically, the outputs of multiple predictive models—including the base model and feature-excluded models—are used as input features, and the meta-model learns the optimal weights for combining these predictions. By leveraging the strengths of each model and compensating for their weaknesses, this tactic enables the system to handle complex distribution shifts that are difficult for a single model to address. Although this approach incurs relatively high computational costs, it achieves more robust predictions.

In summary, the Plan phase achieves high adaptability by integrating two adaptive tactics: model switching and ensemble model building. The model switching tactic is particularly effective in scenarios where a single feature exhibits distribution shifts, while the ensemble model building tactic excels in situations where multiple features shift simultaneously, making adaptation through model switching insufficient. By effectively combining and selecting these two tactics, the Plan phase ensures adaptability to diverse patterns of distribution shifts, even in operational environments where ground-truth data is unavailable and retraining is not feasible.

e: EXECUTE

In the Execute phase, the adaptive action is implemented. Specifically, the system switches to the model selected during the Plan phase.

IV. EXPERIMENT

To validate the effectiveness of the proposed method, a series of experiments were conducted on a regression prediction task. The objective of these experiments was to evaluate whether the proposed method could provide superior predictive accuracy compared to three baseline approaches, particularly in environments where the input data distribution shifts during deployment. The baseline methods include: (1) a traditional single model without adaptation that continues using the model trained with all features regardless of distribution changes, (2) an adaptation method that always switches to an appropriate pre-trained model when distribution changes occur, and (3) an adaptation method that always constructs ensemble models using pre-trained models when distribution changes occur. The performance of the proposed method was assessed using three evaluation metrics: the coefficient of determination (R^2), root mean square error (RMSE), and mean absolute error (MAE).

For the predictive model, XGBoost, a gradient boosting-based regression model, was employed. XGBoost is a high-accuracy algorithm that combines multiple decision trees and

performs particularly well in regression tasks, making it an appropriate choice for this study.

During model training, a base model was first built using all available features. The training process involved cross-validation to evaluate multiple parameter combinations, with grid search used to determine the optimal hyperparameters. Key hyperparameters tuned included the number of decision trees, maximum depth of decision trees, sampling ratio for each decision tree, and learning rate. Additionally, models excluding each individual feature were built and tuned using the same process as the base model. This procedure was repeated for all features, resulting in the training and saving of both the full-feature model and the feature-excluded models.

The system parameters were set with the following baseline configuration: M = 100 (the number of recent test data samples used for prediction interval width calculation), N = 50 (the number of consecutive samples required to confirm a state), $\theta = 0.5$ (the threshold for feature variation detection using the Kolmogorov-Smirnov statistic), and k = 1.1 (the tolerance parameter). Additionally, a 95% prediction interval was used to define the system requirements, the number of similar samples s selected for ensemble model construction was set to 200, and ridge regression was implemented as the meta-model.

The experiments were conducted on both simulation data and real-world data. Specifically, the following three types of experiments were performed:

1) Performance comparison experiment: A comparison of the proposed method with three baseline methods was conducted using the baseline parameter settings.
2) Parameter sensitivity analysis: Performance was evaluated across nine combinations by varying M and N within 50, 100, 125.
3) Trade-off analysis between computational efficiency and prediction performance (real-world data only): The relationship between prediction accuracy and computational cost was evaluated by varying the k parameter within 1.0, 1.1, 1.3, 1.4.

A. EXPERIMENT WITH SIMULATION DATA

To evaluate the stability of the proposed method under distribution shifts during deployment, we conducted experiments in a controlled environment using simulation data. We generated features from normal distributions with specified parameters and created target variables as linear combinations of these features plus normally distributed noise. Specifically, 2000 training data, 500 validation data, and 10000 test data were generated. The target variable y is generated by the following equation:

$$y = 0.5x_1 - 0.3x_2 + 0.4x_3 - 0.3x_4 + 0.6x_5 + \epsilon \quad (1)$$

where ϵ follows a normal distribution with mean 0 and variance 1 ($\epsilon \sim \mathcal{N}(0, 1)$, where $\mathcal{N}(\mu, \sigma^2)$ denotes a normal distribution with mean μ and variance σ^2)

In the training data, each feature follows the following normal distributions:

$$x_1 \sim \mathcal{N}(8, 3^2) \quad (2)$$
$$x_2 \sim \mathcal{N}(12, 4^2) \quad (3)$$
$$x_3 \sim \mathcal{N}(15, 6^2) \quad (4)$$
$$x_4 \sim \mathcal{N}(10, 2^2) \quad (5)$$
$$x_5 \sim \mathcal{N}(5, 5^2) \quad (6)$$

To simulate distribution shifts in input data during deployment, we intentionally introduced distribution shifts in the test data. The details of the data distribution variations in the test data are shown in Table 1.

TABLE 1. Distribution shift settings in test data.

Period (Index)	Target Feature	Shifted Distribution
500-2100	x_2	$\mathcal{N}(3, 2^2)$
4000-5500	x_4	$\mathcal{N}(0, 3^2)$
7500-9000	x_2	$\mathcal{N}(6, 3^2)$
7500-9000	x_3	$\mathcal{N}(5, 5^2)$

Experiments with deliberate variations in normal distribution parameters were inspired by the study in [39]. This experimental setup enables us to evaluate the prediction performance of the proposed method under various distribution shift scenarios, ranging from single feature shifts to simultaneous changes in multiple features.

Figure 5 and Table 2, 3 present the experimental results obtained using simulation data. Figure 5 shows a comparison of prediction performance among four methods using simulation data with 95% confidence intervals. The compared methods are: (1) single model, (2) switching only, (3) ensemble only, and (4) proposed method. The results demonstrate that the proposed method achieved the best performance across all evaluation metrics (R^2, RMSE, MAE). Specifically, the proposed method achieved approximately 0.868 for R^2, 1.613 for RMSE, and 1.239 for MAE, substantially outperforming the other methods. Furthermore, the 95% confidence intervals of the proposed method do not overlap with those of any other method for all evaluation metrics, confirming significant performance improvements.

TABLE 2. Summary of proposed method performance on simulation data.

Index range	Distribution shift	Model	RMSE
0 - 601	-	base_model	1.430
602 - 2192	x_2	excluding-x_2	1.637
2193 - 4103	-	base_model	1.225
4104 - 5594	x_4	excluding-x_4	1.308
5595 - 7623	-	base_model	1.287
7624 - 9075	x_2, x_3	ensemble	2.690
9076 - 9999	-	base_model	1.165

Table 2 presents an overview of the results when the proposed method is applied. In this table, the test data is segmented into intervals based on the distribution change

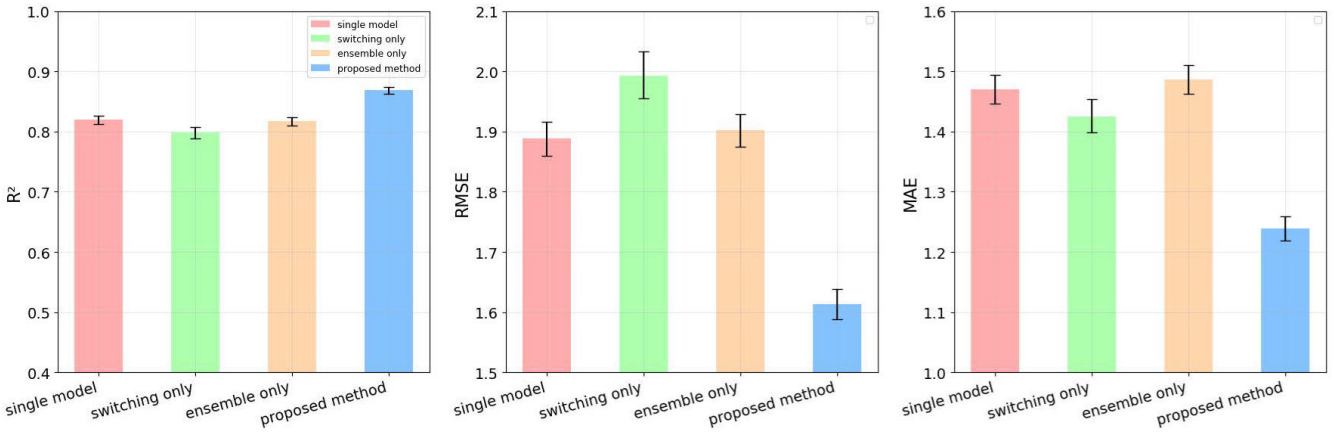

FIGURE 5. Prediction performance results using simulation data with 95% confidence intervals: Single model vs. Switching only vs. Ensemble only vs. Proposed method.

TABLE 3. Summary of single model method on simulation data.

Index range	Distribution shift	Model	RMSE
0 - 601	-	base_model	1.430
602 - 2192	x_2	base_model	2.538
2193 - 4103	-	base_model	1.225
4104 - 5594	x_4	base_model	1.894
5595 - 7623	-	base_model	1.287
7624 - 9075	x_2, x_3	base_model	2.798
9076 - 9999	-	base_model	1.165

points detected by the proposed method, and the results are organized for each interval. This segmentation is not arbitrarily set but is based on the actual boundaries where the data characteristics changed. The drifting features detected in each interval, the models automatically selected by the system, and the RMSE values are compared. Table 3 shows the results of a single model approach that predicts all data using the base model, with the test data segmented in the same manner as Table 2. Comparing the two tables reveals that the proposed method switches the models used in intervals where feature drift occurs. Specifically, in intervals where feature x2 drifts alone, a model trained without using feature x2 is employed. Similarly, in intervals where feature x4 drifts alone, a model trained without using feature x4 is employed. In intervals where features x2 and x3 drift simultaneously, an ensemble model is constructed and used for prediction. Through this autonomous model switching mechanism in intervals where feature drift occurs, improved prediction performance was consistently achieved across all three evaluation metrics (MAE, RMSE, and R^2) in every interval where feature drift was detected.

To evaluate the parameter sensitivity of the proposed method, we conducted experiments using nine different parameter combinations with M ∈ 50, 100, 125 and N ∈ 50, 100, 125 on real-world data. Table 4 shows the performance results across all parameter configurations. The results show that R^2 values ranged from 0.865 to 0.869, RMSE values ranged from 1.607 to 1.631, and MAE values ranged from 1.236 to 1.257. The overlapping 95% confidence intervals across different parameter settings indicate that the proposed method is robust to parameter configuration. Consistently high performance was achieved across all parameter settings, substantially outperforming the single model approach ($R^2 = 0.819$, RMSE $= 1.889$, MAE $= 1.470$).

These simulation experiments demonstrate that the proposed method exhibits superior predictive performance compared to single model approaches and adaptive methods using only a single adaptation tactic, while also possessing stability against parameter variations.

B. EXPERIMENT WITH REAL DATA

In the experiment using real data, we employed the California Housing Dataset, a publicly available dataset widely used in the field of machine learning. This dataset is based on the 1990 U.S. Census data for California and includes information about housing across various regions (districts). Specifically, it contains the features shown in the Table 5, with the target variable being the median house value in each district (measured in tens of thousands of dollars).

To evaluate the effectiveness of the proposed method in handling input data variations, we simulated two types of data distribution shifts. These variations reflect common challenges faced by machine learning systems in real-world operational environments.

First, the initial variation simulated input distribution shifts due to geographical characteristics. Specifically, we split the dataset into southern (low-latitude) and northern (high-latitude) regions using latitude 37° as the boundary. The training data was deliberately biased, with 95% of the samples originating from the southern region. Conversely, the test data included a significant portion of samples from the northern region, particularly in the latter part of the test dataset. This setup enabled the evaluation of performance changes when applying the model to geographical conditions different from those during training. Similar experiments

TABLE 4. Performance comparison of proposed method under different parameter settings on simulation data.

Configuration	R^2 (95% CI)	RMSE (95% CI)	MAE (95% CI)
Single Model	0.819 [0.812, 0.826]	1.889 [1.860, 1.916]	1.470 [1.447, 1.494]
M50-N50	0.869 [0.864, 0.874]	1.608 [1.583, 1.634]	1.236 [1.216, 1.256]
M50-N100	0.869 [0.864, 0.874]	1.607 [1.582, 1.631]	1.237 [1.217, 1.257]
M50-N125	0.869 [0.863, 0.874]	1.612 [1.588, 1.637]	1.243 [1.223, 1.263]
M100-N50	0.868 [0.863, 0.873]	1.614 [1.588, 1.639]	1.239 [1.219, 1.259]
M100-N100	0.868 [0.862, 0.873]	1.617 [1.593, 1.642]	1.246 [1.226, 1.265]
M100-N125	0.866 [0.861, 0.871]	1.627 [1.603, 1.651]	1.254 [1.234, 1.274]
M125-N50	0.869 [0.864, 0.874]	1.609 [1.584, 1.634]	1.238 [1.218, 1.257]
M125-N100	0.866 [0.861, 0.871]	1.625 [1.601, 1.649]	1.252 [1.232, 1.271]
M125-N125	0.865 [0.860, 0.870]	1.631 [1.606, 1.655]	1.257 [1.236, 1.276]

altering regions between training and test datasets have been conducted in previous studies [40], [41]. Such scenarios mimic real-world challenges, such as deploying a real estate price prediction model trained in one region to another region during operation. Additionally, in the input distribution shift due to geographical characteristics, two correlated features representing latitude and longitude vary simultaneously, reflecting the correlated shifts that can occur in real-world scenarios.

Second, the other variation introduced missing values in some input features. Specifically, for certain segments of the test data, the average number of rooms (AveRooms) and population (Population) information were treated as missing values. This simulation reflects scenarios in real-world operational environments where sensor malfunctions, data collection system failures, new regulations, or privacy-related constraints could render specific features unavailable. Previous studies have shown that missing data can adversely affect machine learning model predictions [42], [43]. 10% of the training data, randomly selected, was set aside as validation data. By using test datasets that incorporate these two types of distribution shifts, we can evaluate the model's adaptability to challenges commonly encountered in real-world applications under more realistic conditions.

Figure 6 and Tables 6, 7 present the experimental results using real-world data. Figure 6 shows a comparison of prediction performance among four methods using simulation data with 95% confidence intervals. The compared methods are: (1) single model, (2) switching only, (3) ensemble only, and (4) proposed method. The proposed method also demonstrated superior performance across all evaluation metrics (R^2, RMSE, MAE) compared to other methods on real-world data. The 95% confidence intervals of the proposed method showed no overlap with those of any other method for all evaluation metrics, confirming significant performance improvements on real-world data as well.

Table 6 presents the results of applying the proposed method to real-world data. For real-world data, the test data is also segmented into intervals based on the distribution change points detected by the proposed method, and the results are organized for each interval. The drifting features detected in each interval, the models automatically selected by the system, and the RMSE values are compared. Table 7 shows the results of the single model approach using the same data segmentation. Comparison between these tables confirms that the proposed method appropriately switches models in response to feature drift in real-world data as well. Specifically, in intervals where only feature AveRooms drifts, a model excluding this feature is selected, and in intervals where only feature Population drifts, a model excluding Population is similarly employed. In intervals where features Longitude and Latitude drift simultaneously, an ensemble model is constructed. Through such autonomous model switching, performance improvements were achieved across all three evaluation metrics in every interval where feature drift was detected, even in real-world data.

Parameter sensitivity on real-world data was also evaluated similarly. The experimental results using nine different parameter combinations with $M \in 50, 100, 125$ and $N \in 50, 100, 125$ are shown in Table 8. The results show that R^2 values ranged from 0.793 to 0.797, RMSE values ranged from 0.523 to 0.528, and MAE values ranged from 0.356 to 0.362. The overlapping 95% confidence intervals across different parameter settings indicate the robustness of the proposed method to parameter configurations. Stable high performance was maintained across all parameter settings. These real-world data experiments validated the effectiveness and robustness of the proposed method confirmed in simulations under more realistic conditions.

Next, to evaluate the trade-off between prediction accuracy and computational efficiency, we conducted experiments using different k parameter values (1.0, 1.1, 1.3, 1.4) on real-world data. The k parameter controls the uncertainty tolerance threshold for model selection: lower k values impose stricter requirements on prediction interval width, potentially improving accuracy but increasing computational cost, while higher k values allow greater uncertainty tolerance, reducing processing time but potentially compromising accuracy.

Figure 7 illustrates the RMSE performance across different intervals for each k parameter value. The interval segmentation used here is the same as that in Table 6. In intervals 0-1097, 1098-3098, 3099-4097, and 5599-7487, all k values achieve identical RMSE performance. However, notable differences emerge in intervals 4098-5598 and

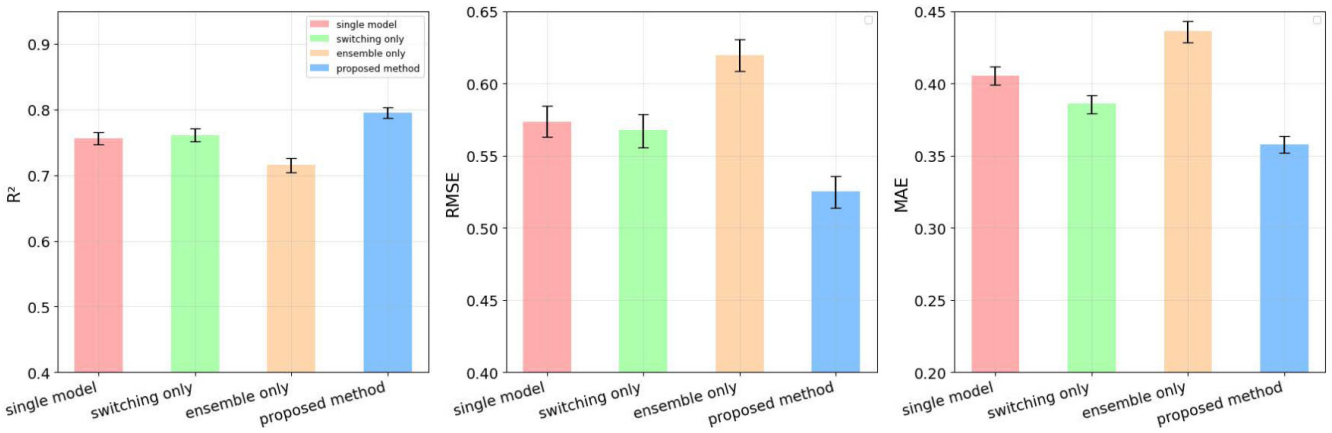

FIGURE 6. Prediction performance results using real data with 95% confidence intervals: Single model vs. Switching only vs. Ensemble only vs. Proposed method.

7488-14639, where lower k values (particularly $k = 1.0$ and $k = 1.1$) demonstrate superior accuracy compared to higher k values.

Table 9 presents the model selection behavior and computational cost for different k parameter values during the entire prediction process. The results demonstrate a clear relationship between the k parameter and both model selection patterns and computational overhead. As the k parameter decreases from 1.4 to 1.0, indicating stricter uncertainty tolerance requirements, the system exhibits more frequent model switching and increased ensemble model construction. Specifically, with $k = 1.4$, the system primarily relies on the base model across most intervals, with only one feature-excluded model (excluding-AveRooms) activated in interval 1098-3098, resulting in the shortest total processing time of 66.5 seconds. In contrast, $k = 1.0$ triggers more complex model selection behavior, including ensemble model construction in intervals 4098-5598 and 7488-14639, leading to the highest computational cost of 160.9 seconds. The intermediate parameter values ($k = 1.1$ and $k = 1.3$) show gradual transitions in model selection complexity, with total processing times of 129.3 and 113.7 seconds, respectively.

TABLE 5. Features of the California housing dataset.

Index	Feature Name	Description
0	MedInc	Median household income in the region
1	HouseAge	Median age of houses in the region
2	AveRooms	Average number of rooms per household
3	AveBedrms	Average number of bedrooms per household
4	Population	Population of the region
5	AveOccup	Average household size
6	Latitude	Latitude of the region
7	Longitude	Longitude of the region

V. DISCUSSION

The experimental results demonstrated that the proposed method improves prediction performance by adapting to data distribution changes even in operational environments where ground-truth data is unavailable. In experiments using simulation data, the proposed method ($R^2 = 0.868$, RMSE = 1.613, MAE = 1.239) achieved substantial performance improvements across all evaluation metrics compared to the single model approach ($R^2 = 0.819$, RMSE = 1.889, MAE = 1.470). Similarly, in experiments using real-world data, the proposed method ($R^2 = 0.797$, RMSE = 0.523, MAE = 0.356) demonstrated superiority over the single model approach ($R^2 = 0.756$, RMSE = 0.573, MAE = 0.405). The absence of overlap in the 95% confidence intervals indicates that these performance improvements are significant.

Furthermore, the superiority of the proposed method was confirmed in comparison with methods employing only a single adaptive tactic. Both the method using only model switching and the method using only ensemble construction showed improved performance over the single model approach; however, neither achieved the performance level of the proposed method. These results suggest that the proposed approach, which possesses multiple adaptive tactics and selects among them according to data distribution

TABLE 6. Summary of proposed method performance on real data.

Index range	Distribution shift	Model	RMSE
0 - 1097	-	base_model	0.520
1098 - 3098	**AveRooms**	excluding-*AveRooms*	0.499
3099 - 4097	-	base_model	0.487
4098 - 5598	**Population**	excluding-*Population*	0.497
5599 - 7487	-	base_model	0.485
7488 - 14639	**Longitude, Latitude**	ensemble	0.553

TABLE 7. Summary of single model performance on real data.

Index range	Distribution shift	Model	RMSE
0 - 1097	-	base_model	0.520
1098 - 3098	**AveRooms**	base_model	0.736
3099 - 4097	-	base_model	0.487
4098 - 5598	**Population**	base_model	0.521
5599 - 7487	-	base_model	0.485
7488 - 14639	**Longitude, Latitude**	base_model	0.572

TABLE 8. Performance comparison of proposed method under different parameter settings on real data.

Configuration	R^2 (95% CI)	RMSE (95% CI)	MAE (95% CI)
Single Model	0.756 [0.746, 0.765]	0.573 [0.563, 0.584]	0.405 [0.399, 0.411]
M50-N50	0.797 [0.789, 0.805]	0.523 [0.512, 0.534]	0.356 [0.350, 0.362]
M50-N100	0.795 [0.787, 0.803]	0.526 [0.514, 0.536]	0.359 [0.353, 0.364]
M50-N125	0.794 [0.786, 0.802]	0.527 [0.516, 0.538]	0.361 [0.355, 0.366]
M100-N50	0.795 [0.788, 0.804]	0.525 [0.514, 0.536]	0.358 [0.352, 0.363]
M100-N100	0.794 [0.786, 0.803]	0.526 [0.515, 0.537]	0.359 [0.353, 0.365]
M100-N125	0.794 [0.786, 0.802]	0.527 [0.516, 0.538]	0.361 [0.355, 0.366]
M125-N50	0.795 [0.787, 0.804]	0.525 [0.514, 0.536]	0.358 [0.352, 0.363]
M125-N100	0.794 [0.786, 0.802]	0.527 [0.515, 0.537]	0.360 [0.353, 0.365]
M125-N125	0.793 [0.785, 0.801]	0.528 [0.517, 0.539]	0.362 [0.356, 0.368]

TABLE 9. Selected models and total model selection time by k parameter.

k parameter	0-1097	1098-3098	3099-4097	4098-5598	5599-7487	7488-14639	Total Time Cost(s)
k=1.4	base_model	excluding-*AveRooms*	base_model	base_model	base_model	base_model	66.5
k=1.3	base_model	excluding-*AveRooms*	base_model	base_model	base_model	ensemble	113.7
k=1.1	base_model	excluding-*AveRooms*	base_model	excluding-*Population*	base_model	ensemble	129.3
k=1.0	base_model	excluding-*AveRooms*	base_model	ensemble	base_model	ensemble	160.9

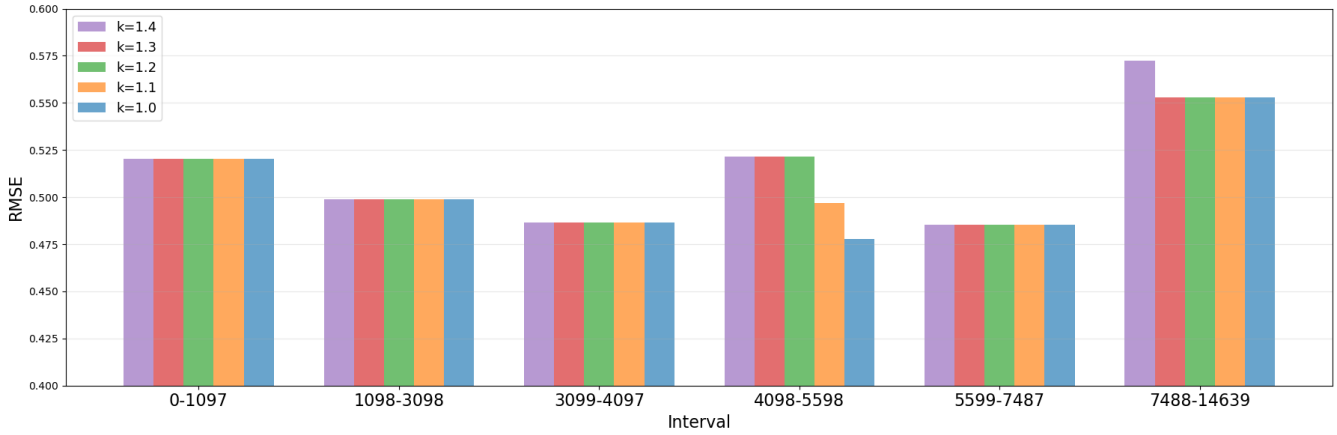

FIGURE 7. RMSE performance across intervals for different k parameters.

changes, is effective. When a single feature shifted, switching to feature-excluded models yielded significant performance improvements, and in intervals where multiple features shifted simultaneously, ensemble models were constructed as existing models could not meet the requirements, achieving certain performance improvements.

Next, we discuss the trade-off between adaptability and computational efficiency controlled by parameter k. As shown in Table 9, parameter k controls the balance between prediction accuracy and computational efficiency through the frequency of adaptive actions. With the strict setting of $k = 1.0$, adaptive actions were executed in four out of six intervals, achieving the highest accuracy but requiring a total processing time of 160.9 seconds. In contrast, with the lenient setting of $k = 1.4$, adaptive actions were executed in only one out of six intervals, resulting in decreased accuracy but reduced total processing time to 66.5 seconds. Intermediate settings of $k = 1.1$ and $k = 1.3$ exhibited different adaptation patterns. With $k = 1.1$, switching to the excluding-Population model was executed in interval 4098-5598, whereas with $k = 1.3$, the base model continued to be used in the same interval. Due to this difference, $k = 1.1$ achieved high accuracy with a total processing time of 129.3 seconds, while $k = 1.3$ achieved moderate accuracy with a total processing time of 113.7 seconds. This flexibility allows systems requiring real-time responses to prioritize processing speed by setting high k values, while batch processing environments can pursue accuracy by setting low k values.

However, the proposed method has a limitation. The adaptability to simultaneous shifts of multiple features was found to be limited. In the simulation data, in the interval

where feature x2 and x3 shifted simultaneously, the ensemble model achieved an RMSE of 2.690, representing only a marginal improvement over the single model's RMSE of 2.798. Similarly, in the real-world data, in the interval where feature Longitude and Latitude shifted simultaneously, the ensemble model achieved an RMSE of 0.553, a slight improvement over the single model's RMSE of 0.572. These results contrast sharply with the substantial performance improvements observed for single feature shifts. For instance, in the simulation data, RMSE improved from 2.538 to 1.637 in the interval where only x2 shifted, and from 1.894 to 1.308 in the interval where only feature x4 shifted. In the real-world data, RMSE improved from 0.736 to 0.499 in the interval where only feature AveRooms shifted, and from 0.521 to 0.497 in the interval where only Population shifted.

The cause of this limited performance improvement can be attributed to the ensemble model construction method in the proposed approach. When multiple features shift simultaneously, a structural problem arises that diminishes the effectiveness of feature-excluded models. For example, when geographically related features such as Longitude and Latitude in the real-world data shift simultaneously, a model excluding only one of them cannot satisfy the prediction interval width requirements because the remaining feature still contains the effects of the distribution shift. Consequently, at the point when ensemble construction is selected, all available pre-trained models have already been judged insufficient. Under such circumstances, dramatic performance improvements cannot be expected even through stacking-based ensemble methods that rely on the same insufficient model pool.

This result suggests a limitation of adaptive methods that rely solely on pre-prepared model pools in operational environments where ground-truth data is unavailable.

VI. CONCLUSION

In this study, we proposed a self-adaptive framework for machine learning systems that can be applied even in scenarios where ground-truth data is unavailable during operation. The proposed method combines dynamic switching to pre-trained models created during the training phase with a tactic for building ensemble models tailored to specific data distributions. This approach enables the system to flexibly adapt to changes in data distribution, even without access to ground-truth data during operation.

Experimental results confirmed that the proposed method showed superior predictive performance compared to conventional single model approaches and methods employing only a single adaptive tactic in scenarios with shifting data distributions. Furthermore, parameter sensitivity analysis demonstrated that the proposed method maintains stable high performance across different parameter settings, indicating that the prediction accuracy of the proposed method does not critically depend on specific parameter configurations. Additionally, it was shown that users can balance the trade-off between prediction accuracy and processing speed by adjusting the system requirement parameters, providing additional flexibility in meeting operational needs.

Future challenges include improving adaptability to shifts in multiple features. To achieve this, it is necessary to develop more advanced adaptation tactics that can be executed even in scenarios where ground-truth data is unavailable during operation. Additionally, exploring methods to enhance adaptability by combining a greater variety of adaptation tactics may also be worthwhile.

Additionally, while the current method allows users to define the acceptable level of prediction uncertainty by setting the width of the prediction interval as a system requirement, the interpretation of this value can be difficult. Determining an appropriate value is often challenging.Therefore, it is essential to develop visualization methods that present these metrics in a more intuitive and user-friendly manner, as well as mechanisms to automatically suggest suitable values for the width of the prediction interval. Such advancements would enhance usability and facilitate the practical application of the system.

Several directions for future work emerge from these findings. First, the computational overhead associated with ensemble model building presents a significant challenge, particularly in time-critical applications where rapid responses are essential. While the current framework provides flexibility through its dual-strategy approach, the time required for ensemble construction may become prohibitive in scenarios with strict latency requirements. To address this limitation, future research should investigate adaptive timeout mechanisms that can intelligently terminate ensemble building processes and default to faster model switching when computational resources or time constraints are exceeded.

Second, the current evaluation has primarily focused on relatively simple distribution shifts, such as monotonic changes in individual features. However, real-world operational environments often present far more complex challenges, including structural changes in feature relationships, non-linear distribution shifts, and interdependent variations across multiple dimensions simultaneously. Consequently, developing robust adaptation mechanisms capable of detecting and responding to these complex patterns represents a critical area for future investigation.

Third, the current framework relies on pre-trained models prepared during the training phase. While this design ensures computational efficiency and system stability during operation, it may limit adaptability to unforeseen distribution shifts. Future work could explore the integration of online fine-tuning mechanisms that dynamically update model parameters during operation, potentially starting from pre-trained models as initial states. Such an approach could enhance the framework's adaptability to complex and evolving distribution shifts.

Finally, the scalability of the proposed framework across different system architectures and data volumes requires systematic evaluation. As machine learning systems are increasingly deployed in diverse environments ranging from

edge devices to large-scale cloud infrastructures, ensuring that the self-adaptive capabilities remain effective across this spectrum is essential for widespread practical adoption. This includes optimizing memory usage, minimizing communication overhead in distributed settings, and maintaining adaptation quality as data volumes scale.

REFERENCES

[1] I. Gerostathopoulos and E. Pournaras, "TRAPPed in traffic? A self-adaptive framework for decentralized traffic optimization," in *Proc. IEEE/ACM 14th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, May 2019, pp. 32–38, doi: 10.1109/SEAMS.2019.00014.

[2] E. Lee, Y.-D. Seo, and Y.-G. Kim, "Self-adaptive framework based on MAPE loop for Internet of Things," *Sensors*, vol. 19, no. 13, p. 2996, Jul. 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:195845038

[3] M. Casimiro, P. Romano, D. Garlan, and L. Rodrigues, "Towards a framework for adapting machine learning components," in *Proc. IEEE Int. Conf. Autonomic Comput. Self-Organizing Syst. (ACSOS)*, Sep. 2022, pp. 131–140.

[4] M. Casimiro, D. Soares, D. Garlan, L. Rodrigues, and P. Romano, "Self-adapting machine learning-based systems via a probabilistic model checking framework," *ACM Trans. Auto. Adapt. Syst.*, vol. 19, no. 3, pp. 1–30, Sep. 2024, doi: 10.1145/3648682.

[5] P. Oreizy, M. Gorlick, R. Taylor, D. Heimhigner, G. Johnson, N. Medvidovic, A. Quilici, D. Rosenblum, and A. Wolf, "An architecture-based approach to self-adaptive software," *IEEE Intell. Syst. Their Appl.*, vol. 14, no. 3, pp. 54–62, 1999.

[6] D. Weyns, M. U. Iftikhar, D. G. de la Iglesia, and T. Ahmad, "A survey of formal methods in self-adaptive systems," in *Proc. 5th Int. C* Conf. Comput. Sci. Softw. Eng.* New York, NY, USA: Association for Computing Machinery, Jun. 2012, pp. 67–79, doi: 10.1145/2347583.2347592.

[7] D. Weyns, *An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective*. Hoboken, NJ, USA: Wiley, 2020.

[8] Y. Brun, G. Di Marzo Serugendo, C. Gacek, H. Giese, H. Kienle, M. Litoiu, H. Müller, M. Pezzè, and M. Shaw, "Engineering self-adaptive systems through feedback loops," in *Software Engineering for Self-Adaptive Systems*, 2009, pp. 48–70. [Online]. Available: https://api.semanticscholar.org/CorpusID:14376429

[9] D. Weyns, B. Schmerl, V. Grassi, S. Malek, R. Mirandola, C. Prehofer, J. Wuttke, J. Andersson, H. Giese, and K. M. Göschka, "On patterns for decentralized control in self-adaptive systems," in *Software Engineering for Self-Adaptive Systems*, 2013, pp. 76–107. [Online]. Available: https://api.semanticscholar.org/CorpusID:10043848

[10] P. Arcaini, E. Riccobene, and P. Scandurra, "Modeling and analyzing MAPE-K feedback loops for self-adaptation," in *Proc. IEEE/ACM 10th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst.*, May 2015, pp. 13–23.

[11] M. Provoost and D. Weyns, "DingNet: A self-adaptive Internet-of-Things exemplar," in *Proc. IEEE/ACM 14th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, May 2019, pp. 195–201.

[12] H. Muccini, R. Spalazzese, M. T. Moghaddam, and M. Sharaf, "Self-adaptive IoT architectures: An emergency handling case study," in *Proc. 12th Eur. Conf. Softw. Archit., Companion* New York, NY, USA: Association for Computing Machinery, Sep. 2018, pp. 1–6, doi: 10.1145/3241403.3241424.

[13] J. Cámara, H. Muccini, and K. Vaidhyanathan, "Quantitative verification-aided machine learning: A tandem approach for architecting self-adaptive IoT systems," in *Proc. IEEE Int. Conf. Softw. Archit. (ICSA)*, Mar. 2020, pp. 11–22.

[14] J. Wuttke, Y. Brun, A. Gorla, and J. Ramaswamy, "Traffic routing for evaluating self-adaptation," in *Proc. 7th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, Jun. 2012, pp. 27–32.

[15] P. H. Maia, L. Vieira, M. Chagas, Y. Yu, A. Zisman, and B. Nuseibeh, "Dragonfly: A tool for simulating self-adaptive drone behaviours," in *Proc. IEEE/ACM 14th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, May 2019, pp. 107–113, doi: 10.1109/SEAMS.2019.00022.

[16] G. Moreno, C. Kinneer, A. Pandey, and D. Garlan, "DARTSim: An exemplar for evaluation and comparison of self-adaptation approaches for smart cyber-physical systems," in *Proc. IEEE/ACM 14th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, May 2019, pp. 181–187, doi: 10.1109/SEAMS.2019.00031.

[17] A. Bennaceur, C. McCormick, J. G. Galán, C. Perera, A. Smith, A. Zisman, and B. Nuseibeh, "Feed me, feed me: An exemplar for engineering adaptive software," in *Proc. 11th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst.* New York, NY, USA: Association for Computing Machinery, May 2016, pp. 89–95, doi: 10.1145/2897053.2897071.

[18] M. U. Iftikhar, G. S. Ramachandran, P. Bollansée, D. Weyns, and D. Hughes, "DeltaIoT: A self-adaptive Internet of Things exemplar," in *Proc. IEEE/ACM 12th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, May 2017, pp. 76–82.

[19] Y.-J. Shin, L. Liu, S. Hyun, and D.-H. Bae, "Platooning LEGOs: An open physical exemplar for engineering self-adaptive cyber-physical systems-of-systems," in *Proc. Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, May 2021, pp. 231–237.

[20] O. Gheibi, D. Weyns, and F. Quin, "Applying machine learning in self-adaptive systems: A systematic literature review," *ACM Trans. Auto. Adapt. Syst.*, vol. 15, no. 3, pp. 1–37, Aug. 2021, doi: 10.1145/3469440.

[21] T. R. D. Saputri and S.-W. Lee, "The application of machine learning in self-adaptive systems: A systematic literature review," *IEEE Access*, vol. 8, pp. 205948–205967, 2020.

[22] P. Jamshidi, A. Sharifloo, C. Pahl, H. Arabnejad, A. Metzger, and G. Estrada, "Fuzzy self-learning controllers for elasticity management in dynamic cloud architectures," in *Proc. 12th Int. ACM SIGSOFT Conf. Quality Softw. Archit. (QoSA)*, Apr. 2016, pp. 70–79.

[23] P. Jamshidi, C. Pahl, and N. C. Mendonça, "Managing uncertainty in autonomic cloud elasticity controllers," *IEEE Cloud Comput.*, vol. 3, no. 3, pp. 50–60, May 2016.

[24] A. B. Ismail and V. Cardellini, "Decentralized planning for self-adaptation in multi-cloud environment," in *Proc. ESOCC Workshops*, 2014. [Online]. Available: https://api.semanticscholar.org/CorpusID:477319

[25] I. Gerostathopoulos, D. Skoda, F. Plasil, T. Bures, and A. Knauss, "Architectural homeostasis in self-adaptive software-intensive cyber-physical systems," in *Proc. Eur. Conf. Softw. Archit.*, Copenhagen, Denmark. Cham, Switzerland: Springer, 2016, pp. 113–128.

[26] I. Elgendi, M. F. Hossain, A. Jamalipour, and K. S. Munasinghe, "Protecting cyber physical systems using a learned MAPE-K model," *IEEE Access*, vol. 7, pp. 90954–90963, 2019.

[27] S. Yamagata, H. Nakagawa, Y. Sei, Y. Tahara, and A. Ohsuga, "Self-adaptation for heterogeneous client-server online games," in *Proc. 18th IEEE/ACIS Int. Conf. Comput. Inf. Sci. (ICIS)*, Jun. 2019, pp. 65–79.

[28] F. Quin, D. Weyns, T. Bamelis, S. Singh Buttar, and S. Michiels, "Efficient analysis of large adaptation spaces in self-adaptive systems using machine learning," in *Proc. IEEE/ACM 14th Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, May 2019, pp. 1–12.

[29] M. Sommer, S. Tomforde, J. Hahner, and D. Auer, "Learning a dynamic re-combination strategy of forecast techniques at runtime," in *Proc. IEEE Int. Conf. Autonomic Comput.*, Jul. 2015, pp. 261–266.

[30] A. Pandey, B. Schmerl, and D. Garlan, "Instance-based learning for hybrid planning," in *Proc. IEEE 2nd Int. Workshops Found. Appl. Self* Syst. (FAS*W)*, Sep. 2017, pp. 64–69.

[31] S. Baldi, G. Battistelli, E. Mosca, and P. Tesi, "Multi-model unfalsified adaptive switching supervisory control," *Automatica*, vol. 46, no. 2, pp. 249–259, Feb. 2010. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0005109809004956

[32] G. Battistelli, E. Mosca, M. G. Safonov, and P. Tesi, "Stability of unfalsified adaptive switching control in noisy environments," *IEEE Trans. Autom. Control*, vol. 55, no. 10, pp. 2424–2429, Oct. 2010.

[33] Z. Han and K. S. Narendra, "New concepts in adaptive control using multiple models," *IEEE Trans. Autom. Control*, vol. 57, no. 1, pp. 78–89, Jan. 2012.

[34] S. Baldi, P. Ioannou, and E. Mosca, "Multiple model adaptive mixing control: The discrete-time case," *IEEE Trans. Autom. Control*, vol. 57, no. 4, pp. 1040–1045, Apr. 2012.

[35] M. Casimiro, P. Romano, D. Garlan, G. A. Moreno, E. Kang, and M. Klein, "Self-adaptation for machine learning based systems," in *Proc. Eur. Conf. Softw. Archit.*, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:239040504

[36] M. Casimiro, P. Romano, D. Garlan, G. A. Moreno, E. Kang, and M. Klein, "Self-adaptive machine learning systems: Research challenges and opportunities," in *Proc. Eur. Conf. Softw. Archit.* Cham, Switzerland: Springer, 2021, pp. 133–155.

[37] S. Kulkarni, A. Marda, and K. Vaidhyanathan, "Towards self-adaptive machine learning-enabled systems through QoS-aware model switching," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2023, pp. 1721–1725.
[38] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241–259, Jan. 1992. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0893608005800231
[39] T. Zhang, I. Yamane, N. Lu, and M. Sugiyama, "A one-step approach to covariate shift adaptation," *Social Netw. Comput. Sci.*, vol. 2, no. 4, Jun. 2021, doi: 10.1007/s42979-021-00678-6.
[40] X. Chen, M. Monfort, A. Liu, and B. D. Ziebart, "Robust covariate shift regression," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:13893815
[41] A. J. Storkey and M. Sugiyama, "Mixture regression for covariate shift," in *Proc. 19th Int. Conf. Neural Inf. Process. Syst.* Cambridge, MA, USA: MIT Press, 2006, pp. 1337–1344.
[42] S. Schelter, T. Rukat, and F. Biessmann. (2021). *Jenga: A Framework to Study the Impact of Data Errors on the Predictions of Machine Learning Models*. [Online]. Available: https://www.amazon.science/publications/jenga-a-framework-to-study-the-impact-of-data-errors-on-the-
[43] C. Batini and M. Scannapieco, *Data Quality: Concepts, Methodologies and Techniques (Data-Centric Systems and Applications)*. Berlin, Germany: Springer-Verlag, 2006.

KENTO FURUKAWA received the bachelor's degree from the Department of Computer Science and Systems Engineering, Faculty of Engineering, Kobe University, in 2023. He is currently pursuing the degree with the Graduate School of Information Science and Technology, Osaka University.

HIROYUKI NAKAGAWA (Member, IEEE) received the B.S. degree in computer science from Osaka University, in 1997, the M.S. degree in computer science from The University of Tokyo, in 2007, and the Ph.D. degree in computer science from Waseda University, in 2013. From 1997 to 2008, he was with Kajima Corporation. From 2008 to 2013, he was an Assistant Professor with The University of Electro-Communications. From 2014 to 2024, he was an Assistant Professor with Osaka University. He is currently a Professor with Okayama University and an Invited Professor with Osaka University. His research interests include self-adaptive systems, requirements engineering, software evolution, and probabilistic model checking.

TATSUHIRO TSUCHIYA (Member, IEEE) received the M.E. and Ph.D. degrees from Osaka University, in 1995 and 1998, respectively. He is currently a Professor with the Department of Information Systems Engineering, Osaka University. His research interests include model checking, software testing, dependable systems, and autonomous systems.